

```
#include <iostream>

#include <stdexcept>

template <typename T>
class DoubleLinkedList {
private:
    // Структура вузла списку
    struct Node {
        T data;
        Node* next;
        Node* prev;

        Node(const T& item) : data(item), next(nullptr), prev(nullptr) {}
    };

    Node* head; // Вказівник на перший елемент
    Node* tail; // Вказівник на останній елемент
    size_t size; // Розмір списку

public:
    // Конструктор
    DoubleLinkedList() : head(nullptr), tail(nullptr), size(0) {}

    // Деструктор для очищення пам'яті
    ~DoubleLinkedList() {
        clear();
    }

    // Додавання елемента в початок списку
    void push_front(T item) {
        Node* newNode = new Node(item);
        if (head == nullptr) { // Якщо список порожній
```

```
        head = tail = newNode;
    } else {
        newNode->next = head;
        head->prev = newNode;
        head = newNode;
    }
    size++;
}
```

// Додавання елемента в кінець списку

```
void push_back(T item) {
    Node* newNode = new Node(item);
    if (tail == nullptr) { // Якщо список порожній
        head = tail = newNode;
    } else {
        newNode->prev = tail;
        tail->next = newNode;
        tail = newNode;
    }
    size++;
}
```

// Видалення першого знайденого елемента за значенням

```
void remove(const T& item) {
    Node* current = head;

    while (current != nullptr) {
        if (current->data == item) {
            // Якщо це єдиний елемент у списку
            if (current == head && current == tail) {
                head = tail = nullptr;
            }
        }
    }
}
```

```

        // Якщо це перший елемент
        else if (current == head) {
            head = head->next;
            head->prev = nullptr;
        }

        // Якщо це останній елемент
        else if (current == tail) {
            tail = tail->prev;
            tail->next = nullptr;
        }

        // Якщо елемент всередині списку
        else {
            current->prev->next = current->next;
            current->next->prev = current->prev;
        }

        delete current;

        size--;

        return; // Видаляємо лише перший знайдений і виходимо
    }

    current = current->next;
}
}

```

```

// Отримання посилання на перший елемент
T& front() const {
    if (head == nullptr) {
        throw std::underflow_error("Список порожній!");
    }

    return head->data;
}

```

```
// Отримання посилання на останній елемент
T& back() const {
    if (tail == nullptr) {
        throw std::underflow_error("Список порожній!");
    }
    return tail->data;
}
```

```
// Допоміжний метод для перевірки розміру
size_t getSize() const {
    return size;
}
```

```
// Допоміжний метод для очищення списку
void clear() {
    Node* current = head;
    while (current != nullptr) {
        Node* nextNode = current->next;
        delete current;
        current = nextNode;
    }
    head = tail = nullptr;
    size = 0;
}
```

```
// Метод для виведення списку (для демонстрації)
void print() const {
    Node* current = head;
    while (current != nullptr) {
        std::cout << current->data << " ";
        current = current->next;
    }
}
```

```
std::cout << std::endl;  
}  
};
```